プログラミングワークショップ 2

青森大学ソフトウェア情報学部

橋本恭能

haship@aomori-u.ac.jp

講義資料について

- この講義で使う資料やプリントは以下のWebページから閲覧できます
- ブラウザを開き、以下のアドレスを入力してください

URL http://haship.github.io

- Webページ上部の【講義資料】をクリック
- 表から【プログラミングワークショップ２】をクリック
- 目次ページをブックマークする

目次

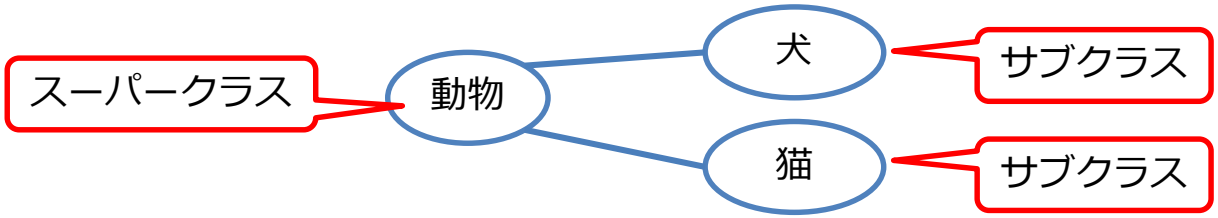
- テーマ クラスの継承3
- 例題
  - メソッドのオーバーライド
- 課題
  - 壁/反射板/ブロッククラス(つづき)
  - Blockクラスのメソッドをオーバーライドを使って実装する

例題 犬クラス・猫クラス・動物クラス

- Dogクラス
  - フィールド
    - 年齢 `int age`
  - メソッド
    - 食べる
      `void eat()`
    - 寝る
      `void sleep()`
    - 吠える(犬クラス独自)
      `void bark()`
- Catクラス
  - フィールド
    - 年齢 `int age`
  - メソッド
    - 食べる
      `void eat()`
    - 寝る
      `void sleep()`
    - 鳴く(猫クラス独自)
      `void meow()`

動物クラス

- 犬・猫クラスの共通点を「動物」クラスとして定義
  - 共通するフィールドやメソッドは、動物クラスで定義、犬・猫クラスは動物クラスのフィールドやメソッドを引き継ぐ
    - スーパークラス（親クラス）：継承元のクラス
    - サブクラス（子クラス）：継承した新しいクラス



Animalクラス

```
class Animal {

  int age;                    //年齢は犬・猫クラス共通なので、動物クラスで定義

  Animal(){                   //コンストラクタ
    age = 0;                  //生成したら年齢は0に初期化
  }

  void eat(){                 //食べるは犬・猫クラス共通なので、動物クラスで定義
    println("Yum!");
  }

  void sleep(){               //寝るは犬・猫クラス共通なので、動物クラスで定義
    println("zzz...");
  }

}
```

Animalクラスを継承したDogクラス

```
class Dog extends Animal {

  Dog(){
    super();                    //スーパークラスのコンストラクタを実行
  }

  void bark(){                  //吠えるは犬クラス独自のメソッド
    println("WOOF!");
  }

}
```

スーパークラスAnimalを継承する

共通するフィールドやメソッドは
Animalクラスで定義しているため
Dogクラスから削除

Animalクラスを継承したCatクラス

```
class Cat extends Animal {

  Cat(){
    super();                    //スーパークラスのコンストラクタを実行
  }

  void meow(){                  //鳴くは猫クラス独自のメソッド
    println("MEOW");
  }

}
```

スーパークラスAnimalを継承する

共通するフィールドやメソッドは
Animalクラスで定義しているため
Catクラスから削除

メインタブ

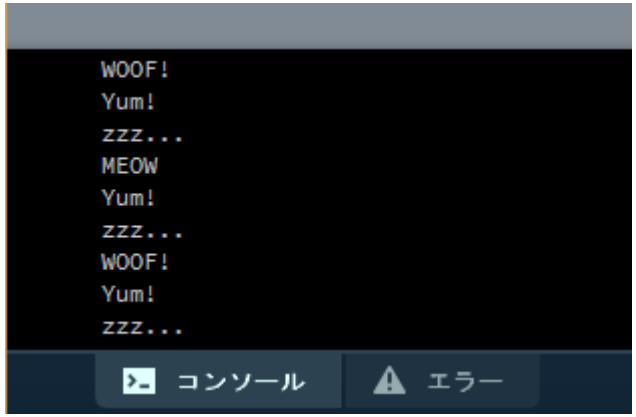
```
Dog dog;              //犬オブジェクト
Cat cat;              //猫オブジェクト

void setup(){
  dog = new Dog();
  cat = new Cat();
}

void draw(){
  dog.eat();                    //犬が食べる
  dog.sleep();                  //犬が寝る
  dog.bark();                   //犬が吠える

  cat.eat();                    //猫が食べる
  cat.sleep();                  //猫が寝る
  cat.meow();                   //猫が鳴く

}
```



メソッドをオーバーライドする

- スーパークラスのメソッドをサブクラスで定義しなおすことができる
- 共通のメソッドだったeat()を猫クラスのメソッドとして再定義
  - 食べる void eat()

```
void eat(){
  println(” Oops” );
}
```

文字列を変更

Catクラスのメソッドをオーバーライド

```
class Cat extends Animal {

  Cat(){
    super();                    //スーパークラスのコンストラクタを実行
  }

  void eat(){                   // 食べる（オーバーライド）
    println("Oops");
  }

  void meow(){                  //鳴くは猫クラス独自のメソッド
    println("MEOW");
  }

}
```

スーパークラスAnimalを継承する

eat()メソッドをオーバーライドで再定義

メインタブ（変更なし）

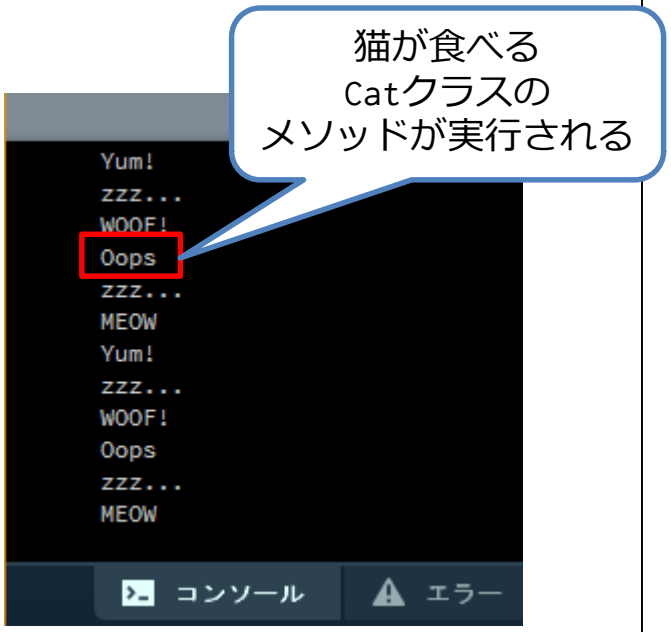
```
Dog dog;              //犬オブジェクト
Cat cat;              //猫オブジェクト

void setup(){
  dog = new Dog();
  cat = new Cat();
}

void draw(){
  dog.eat();                    //犬が食べる
  dog.sleep();                  //犬が寝る
  dog.bark();                   //犬が吠える

  cat.eat();                    //猫が食べる
  cat.sleep();                  //猫が寝る
  cat.meow();                   //猫が鳴く

}
```



課題10 オーバーライド

- Kadai_10
  - 壁・反射板・ブロッククラスを実装して動作確認する
  - ブロッククラスのメソッドをオーバーライドを使って実装する

壁・反射板・ブロックの共通点

- 壁クラスを追加する
  - フィールド
    - x座標、y座標、幅、高さ
  - メソッド
    - ボールと(左・右・上・下)側で衝突したかどうか
    - ボールと頂点で衝突したかどうか
    - 描く

壁・反射板・ブロッククラスの共通点

		反射板	ブロック	壁	共通点
フィールド	x座標、y座標	○	○	○	○
	幅、高さ	○	○	○	○
	速度	○	-	-	
	見えているかどうか	-	○	-	
メソッド	コンストラクタ	×	×	×	
	移動する	○	-	-	
	消える	-	○	-	
	描く	○	△	○	○
	ボールと(左右上下)側で衝突するかどうか	○	△	○	○
	ボールと頂点で衝突するかどうか	○	△	○	○

名前が異なる

名前は共通
処理が異なる

「四角いもの」クラス

- 共通点を「四角いもの」クラス（クラス名 Rect）として定義
- 反射板・壁・ブロックは「四角いもの」クラスを継承する
- 共通するフィールドやメソッドを「四角いもの」クラスで定義する

Rectクラス

```
class Rect {

  float x;          // x座標
  float y;          // y座標
  float w;          // 幅
  float h;          // 高さ

  // コンストラクタ
  Rect(float tempX, float tempY, float tempW, float tempH) {
    x = tempX;
    y = tempY;
    w = tempW;
    h = tempH;
  }

  // 描く
  void display() {
    rect(x, y, w, h);
  }
```

Rectクラス

```
// ボールと頂点で衝突したかどうか
boolean isHitCorner(Ball b) {
  if (dist(b.x, b.y, x, y) < b.radius ||
    dist(b.x, b.y, x, y+h) < b.radius ||
    dist(b.x, b.y, x+w, y) < b.radius ||
    dist(b.x, b.y, x+w, y+h) < b.radius) {
    return true;
  } else {
    return false;
  }
}

// ボールと上側で衝突したかどうか
boolean isHitTopSide(Ball b) {
  if (b.x > x && b.x < x+w && dist(b.x, b.y, b.x, y) < b.radius){
    return true;
  } else {
    return false;
  }
}
```

Rectクラス

```
// ボールと下側で衝突したかどうか
boolean isHitBottomSide(Ball b) {
  if (b.x > x && b.x < x+w && dist(b.x, b.y, b.x, y+h) < b.radius){
    return true;
  } else {
    return false;
  }
}

// ボールと左側で衝突したかどうか
boolean isHitLeftSide(Ball b) {
  if (b.y > y && b.y < y+h && dist(b.x, b.y, x, b.y) < b.radius) {
    return true;
  } else {
    return false;
  }
}
```

Rectクラス

```
  // ボールと右側で衝突したかどうか
  boolean isHitRightSide(Ball b) {
    if (b.y > y && b.y < y+h && dist(b.x, b.y, x+w, b.y) < b.radius){
      return true;
    } else {
      return false;
    }
  }

}
```

		共通点
フィールド	x座標、y座標	○
	幅、高さ	○
	速度	
	見えているかどうか	
メソッド	コンストラクタ	×
	移動する	
	消える	
	描く	○
	ボールと(左右上下)側で衝突するかどうか	○
	ボールと頂点で衝突するかどうか	○

Rectを継承したBarクラス

```
//反射板クラス（サブクラス）

class Bar extends Rect {

  float speed = 1.0;    // 速度（初期値 1.0）

  // コンストラクタ
  Bar(float tempX, float tempY) {
    super(tempX, tempY, 80, 20);    // スーパークラスのコンストラクタを使って初期化
  }

  // 移動する
  void move() {
    if (keyPressed && key == CODED) {
      if (keyCode == LEFT) {
        x -= speed;
      } else if (keyCode == RIGHT) {
        x += speed;
      }
    }
    x = constrain(x, 0, width - w);
  }
}
```

		反射板	共通点
フィールド	x座標、y座標	○	○
	幅、高さ	○	○
	速度	○	
	見えているかどうか	-	
メソッド	コンストラクタ	×	
	移動する	○	
	消える	-	
	描く	○	○
	ボールと(左右上下)側で衝突するかどうか	○	○
	ボールと頂点で衝突するかどうか	○	○

Rectを継承したWallクラス（追加）

```
//壁クラス（サブクラス）

class Wall extends Rect {

  // コンストラクタ
  Wall(float tempX, float tempY, float tempW, float tempH){
    super(tempX, tempY, tempW, tempH);  //スーパークラスのコンストラクタを使用して初期化
  }

}
```

		壁	共通点
フィールド	x座標、y座標	○	○
	幅、高さ	○	○
	速度	-	
	見えているかどうか	-	
メソッド	コンストラクタ	×	
	移動する	-	
	消える	-	
	描く	○	○
	ボールと(左右上下)側で衝突するかどうか	○	○
	ボールと頂点で衝突するかどうか	○	○

Rectを継承したBlockクラス

```
class Block extends Rect {

  boolean visible;      // 見えているかどうか

  Block(float tempX, float tempY) {
    super(tempX, tempY, 80, 20);    // スーパークラスのコンストラクタを使用して初期化
    visible = true;                 // 見えている状態にする
  }

  // 描く（オーバーライド）
  void display() {
    if (visible) {      //見えている場合は
      super.display();  //スーパークラスのメソッドdisplay()を呼び出す
    }
  }

  // 消える
  void remove() {
    visible = false;
  }
```

		ブロック	共通点
フィールド	x座標、y座標	○	○
	幅、高さ	○	○
	速度	-	
	見えているかどうか	○	
メソッド	コンストラクタ	×	
	移動する	-	
	消える	○	
	描く	△	○
	ボールと(左右上下)側で衝突するかどうか	△	○
	ボールと頂点で衝突するかどうか	△	○

Rectを継承したBlockクラス

```
// ボールと頂点で衝突したかどうか(オーバーライド)
boolean isHitCorner(Ball b) {
  if(visible && super.isHitCorner(b)) {//スーパークラスのメソッドisHitCorner()を呼び出す
    return true;
  } else {
    return false;
  }
}

// ボールと上側で衝突したかどうか(オーバーライド)
boolean isHitTopSide(Ball b) {
  if (visible &&          ?          ) {
    return true;
  } else {
    return false;
  }
}
```

スーパークラスのメソッド
isHitTopSide()を呼び出す

		ブロック	共通点
フィールド	x座標、y座標	○	○
	幅、高さ	○	○
	速度	-	
	見えているかどうか	○	
メソッド	コンストラクタ	×	
	移動する	-	
	消える	○	
	描く	△	○
	ボールと(左右上下)側で衝突するかどうか	△	○
	ボールと頂点で衝突するかどうか	△	○

Rectを継承したBlockクラス

```
// ボールと下側で衝突したかどうか(オーバーライド)
boolean isHitBottomSide(Ball b) {
  if (visible &&  ?  ) {
    return true;
  } else {
    return false;
  }
}

// ボールと左側で衝突したかどうか(オーバーライド)
boolean isHitLeftSide(Ball b) {
  if (visible &&  ?  ) {
    return true;
  } else {
    return false;
  }
}
```

スーパークラスのメソッド
isHitBottomSide()を呼び出す

スーパークラスのメソッド
isHitLeftSide()を呼び出す

		ブロック	共通点
フィールド	x座標、y座標	○	○
	幅、高さ	○	○
	速度	-	
	見えているかどうか	○	
メソッド	コンストラクタ	×	
	移動する	-	
	消える	○	
	描く	△	○
	ボールと(左右上下)側で衝突するかどうか	△	○
	ボールと頂点で衝突するかどうか	△	○

Rectを継承したBlockクラス

```
  // ボールと右側で衝突したかどうか(オーバーライド)
  boolean isHitRightSide(Ball b) {
    if (visible &&  [ ? ] ){
      return true;
    } else {
      return false;
    }
  }

}
```

スーパークラスのメソッド
isHitRightSide()を呼び出す

		ブロック	共通点
フィールド	x座標、y座標	○	○
	幅、高さ	○	○
	速度	-	
	見えているかどうか	○	
メソッド	コンストラクタ	×	
	移動する	-	
	消える	○	
	描く	△	○
	ボールと(左右上下)側で衝突するかどうか	△	○
	ボールと頂点で衝突するかどうか	△	○

Ballクラス

```
class Ball {
  int radius = 10;
  float x;
  float y;
  float vx;
  float vy;
  int xdir = 1;
  int ydir = 1;

  Ball(float tempX, float tempY,
         float tempVx, float tempVy){
    x = tempX;
    y = tempY;
    vx = tempVx;
    vy = tempVy;
  }

  void move() {
    x += vx * xdir;
    y += vy * ydir;
  }

  void flipXdir() {
    xdir = -xdir;
  }

  void flipYdir() {
    ydir = -ydir;
  }

  void display() {
    ellipse(x, y, radius, radius);
  }

  // ボールと壁の衝突したかどうかのメソッドは
  // Wallクラスのメソッドを追加したので
  // Ballクラスから削除

}
```

メインタブ

```
Ball  ball;          //ボール
Bar   bar;           //反射板
Block block;         //ブロック
Wall  topWall, bottomWall, leftWall, rightWall;     //上壁、下壁、左壁、右壁

void setup() {
  size(400, 400);
  ellipseMode(RADIUS);
  ball = new Ball(110, 60, 0.5, 0.3);               //ボール（110, 60）速度(0.5, 0.3)
  bar = new Bar(width/2, height*0.8);               //反射板（width/2, height*0.8)
  block = new Block(40, height*0.2);                //ブロック（40, height*0.2)
  topWall    = new Wall(0, 0, width, 0);            //上壁（0.0）幅width, 高さ0
  bottomWall = new Wall(0, height, width, 0);       //下壁（0.height）幅width, 高さ0
  leftWall   = new Wall(0, 0, 0, height);           //左壁（0.0）幅0, 高さheight
  rightWall  = new Wall(width, 0, 0, height);       //右壁（width.0）幅0, 高さheight
}
```

メインタブ

```
void draw() {
  background(0);

  ball.move();
  bar.move();

  if (leftWall.isHitRightSide(ball) || rightWall.isHitLeftSide(ball)) {
    ball.flipXdir();
  }
  if (topWall.isHitBottomSide(ball) || bottomWall.isHitTopSide(ball)) {
    ball.flipYdir();
  }

  if(bar.isHitTopSide(ball) || bar.isHitBottomSide(ball)){
    ball.flipYdir();
  }
  if(bar.isHitRightSide(ball) || bar.isHitLeftSide(ball)){
    ball.flipXdir();
  }
  if(bar.isHitCorner(ball)){
    ball.flipXdir();
    ball.flipYdir();
  }
```

ボール、反射板の座標の更新

壁とボールの衝突判定

反射板とボールの衝突判定

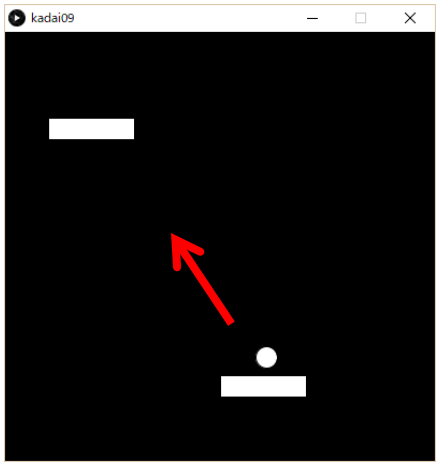
メインタブ

```
  if(block.isHitTopSide(ball) || block.isHitBottomSide(ball)){
    ball.flipYdir();
    block.remove();
  }
  if(block.isHitRightSide(ball) || block.isHitLeftSide(ball)){
    ball.flipXdir();
    block.remove();
  }
  if(block.isHitCorner(ball)){
    ball.flipXdir();
    ball.flipYdir();
    block.remove();
  }

  ball.display();
  topWall.display();
  bottomWall.display();
  leftWall.display();
  rightWall.display();
  bar.display();
  block.display();
}
```

ブロックとボールの衝突判定

表示

動作確認

- ボールが壁で反射するかどうか
- ボールが反射板で反射するかどうか
- ボールがブロックで反射するかどうか
- ボールがブロックに衝突したらブロックが消えるか
- ブロックのあった場所をボールが通過しても反射しないか